

# Project Neo:

## Enable *Heinz* to dodge Bullets like the chosen one

Dominic Zahn

**Abstract**—With robots like the H1 from Unitree, robotics moves in a direction to move on from push recovery towards dodge evasion. While past robotic generations were dependent on recovering from small perturbations, the new generation is agile and stable enough to evade such pushes.

Dodge motions, however, bring multiple hurdles with them. They demand real-time computation of stable dodge poses paired with a robotics platform that is capable of employing potentially dynamic stability. To push the dodging research further, Project Neo aims to provide a starting point.

A portable and easy-to-use framework with an extensive toolbox towards optimization-based dodge control is presented. Utilizing the constrained nature of the dodge problem to find mathematically optimized solutions.

The system uses well-established libraries like Rigid Body Dynamics Library (RBDL), NLOpt, and Eigen3 as foundations. For fast prototyping and ease-of-use, Robot Operating System 2 (ROS2) and Gazebo are preset up inside a Docker container. Added on top are custom packages to further improve the ability to develop new algorithms to solve the dodge problem. All of it is freely available and open source on GitHub. (link to repository)

### I. INTRODUCTION

During the last few years, humanoids have become stronger, more stable, and physically more capable of recovering from perturbations. Robots like those from Unitree or Boston Dynamics can overcome challenges that their predecessors could only dream of.

These capabilities enable new motions and maneuvers to be explored. While push recovery is a valid post-impact strategy, a much better opportunity would be to avoid the impact itself. Common strategies for perturbation compensation or push recovery consist of ankle, hip, or suspensory counteractions. [1] Those, however, are only feasible for small forces. In cases where such forces are exceeded, a capture step is necessary. [2]

A far superior solution would be to dodge the incoming impact. This way the humanoid can calculate beforehand the resulting motion as well as counteract the possible stability problems. Having such powerful platforms at hand enables research towards extended dodge maneuvers. These motions were previously not feasible due to the lack of power, stability, and joint range.

#### A. Dodge Formulation

The idea of moving out of the way of an incoming object or projectile is interesting for multiple reasons. First, there is the obvious limitation of physical damage. Not being hit by

a rapidly approaching obstacle avoids hardware damage and possible sensor misplacements. When thinking of dodging an approaching obstacle, not only airborne objects are to be considered. Scenarios where humanoids work together with, for example, robot arms or other industrial-grade machines can lead to the same situation. All these fall under the following problem formulation:

*A humanoid has to avoid contact with a rapidly approaching object.*

It also serves as a test bed for controlling the humanoid on the edge of stability. The rapid nature and leaned-over poses push the robot to its limits. From an academic standpoint, this state enables different kinds of stability to be examined and evaluated.

This project will mainly focus on static-avoiding actions. The humanoid is not allowed to move from its starting position and therefore needs to lean or duck to move its body away from the object. This strategy will be called *dodge*, while a strategy that changes the position of the robot and moves it to a great degree away from the starting position will be called *avoid*. An example of the latter could be the work of Cognetti et al. in [3].

To create a dodge motion on a humanoid platform, three steps are required:

- 1) Track approaching Object
- 2) Discover existence of statically stable Dodge Pose
- 3) Move towards Pose in a stable Motion

The work of this paper mainly concentrates on the second step while enabling the first and third steps through providing a convenient framework.

Detecting and tracking objects is a common problem in computer vision. It can look back at a long history of research with multiple different approaches. [4]

For the last step, optimal control was chosen over a learning-based solution.

#### B. Optimal Control and Optimization

While most of the current research in this area focuses on machine learning [5] [6], optimal control is well suited to the nature of the problem. Through the use of hard constraints, a solution can be ensured that is inside the specified time span and avoids a collision.

In the example of calculating the necessary controls for a diver, the authors of [7] show the capabilities of optimal control. They display how the precise formulation of a high-complexity problem leads to a highly constrained and specific motion without predefined knowledge of the human

action. Based on this research, the assumption can be made that optimal control is capable of solving the control problem of finding and moving towards an optimal dodge position. As a result, the discovery step (2) and motion step (3) can be solved by one optimal control problem. Due to time constraints, only step two was solved with a simple optimization problem, like it will be described in section II.

### C. Related Work

Looking towards already existing research about dodging robots, a multitude of papers about usage on Unmanned Aerial Vehicle (UAV) are found. [8] [9] [10] Because of their relatively easy inverse kinematics and full 3D movability, those are complexity-wise a lot easier to move away from approaching objects than humanoids. Another advantage of UAVs is their small hitbox and fast reaction speed. So most of those papers focus on controlling whole swarms of flying robots or their lack of computation paired with error-prone sensors. These two problems are not as oppressive in humanoids. Due to their size, they are often equipped with computationally stronger processing units. These days, most humanoids come with Light Detection and Ranging (LiDAR) and camera sensors to enable sensor fusion. Works like [8], [9] and [10] rely on learning-based methods to achieve fast reaction times on little hardware. A solution that is feasible for robots with small Degree Of Freedom (DoF). Because of their low DoF, robot arms are also popular. Works like [5] and even the foundations laying [11] demonstrate their work on a 7-DoF-Arm.

While humanoids have been able to learn complex controls [12] [13], a full learning-based approach cannot ensure presented dodge requirements.

In contrast, papers on humanoids are mostly about footstep planning and creating human-like avoidance trajectories in a macro perspective. [14] [15] [16] Robots are mostly trying to move around a big obstacle or squeeze through tight spaces. [3] [17] In these scenarios the environment is mostly static or changes slowly. The focus of this research lies in complex environment navigation, not whole-body locomotion to achieve precise dodge maneuvers.

Another common topic is humanoids, which are evading humans or other robots. [15] Additionally, works like [17] concentrate their efforts on specifying one main task, which shares attention with the collision avoidance part of their framework.

In contrast this project investigates the motion of dodging as its primary task. The presented novelty of this report, therefore, lies in its focus on the dodge motion and the idea of creating a foundation to encourage in-depth research in this direction.

### D. Overview

As the target platform for this paper, the humanoid H1 from Unitree was chosen. Multiple articles have shown the capabilities of this robot in pushing towards difficult poses. [18] [19] His strong actuators and lightweight body lend themselves towards the dodging paradigm.

To fully explore the capabilities of the H1 in regard to dodge motions, a framework is required. Most of the already existing frameworks are based on Multi-Joint dynamics with Contact (MuJoCo) [20] and targeted towards learning-based approaches.

However, as already discussed in the previous section, the problem formulation of dodging on a humanoid lends itself perfectly to a mathematically optimized solution.

To encourage further research towards an optimization-based solution, a toolbox is built around ROS2 Jazzy and the simulation software Gazebo. [21] As supporting structure, a multitude of libraries is already set up and documented through a simple but effective optimization approach.

The structure of this report is as follows. To showcase and further specify the general problem of dodging objects, a basic optimization problem is formulated in section II.

Based on the found components and possible pitfalls of such a problem, the toolbox itself is presented. Section III functions, therefore, as an implementation and design overview. It enables the reader to utilize the presented tools and describes their possible use cases. Starting with an introduction of the foundational libraries and technologies, which are included and ready to use inside the presented framework. To put these different elements into a bigger picture, an architectural overview is laid out. It displays the dependencies and interfaces between each tool. At the end of this section, the RBDL Wrapper is presented as a bridge between simulation and dynamics library. The functionality of the presented wrapper lies mainly, but not only, in its ability to keep Rigid Body Dynamics Library (RBDL) synced with the simulation. To get quick and seamless access to the dynamics and kinematics of the robot, a tool like RBDL is necessary. However, because the library was mostly developed for offline computations, a synchronization with published joint states from Gazebo is required.

Based on the in section II defined optimization problem an example implementation is presented in section IV. It also showcases common pitfalls when working with mathematical optimization on a humanoid with high DoF. During this section, the reader gets familiar with the presented tools and how to utilize them to evade said pitfalls.

The final chapter (V) wraps up the report with a conclusion of the presented implementation, as well as the capabilities of the framework.

## II. BASIC OPTIMIZATION PROBLEM

The following section outlines a general optimization problem for the beforehand-presented dodge question. First we take a look at the to this project specific (eq. 1) implementation.

Using this specification, a more general description is derived. It shows off each essential component and discusses some possibilities.

The exemplary optimization problem for finding a stati-

cally stable dodge pose is as follows:

$$\xrightarrow{\text{static}} p_{\text{CoP}} = \begin{bmatrix} p_{\text{CoM}} \cdot x \\ p_{\text{CoM}} \cdot y \\ 0 \end{bmatrix} \quad (0a)$$

$$C_{\text{PoS}} := \text{PoS Centroid} \quad (0b)$$

$$\underset{q}{\text{minimize}} \quad s(q) = \|C_{\text{PoS}} - p_{\text{CoP}}\| \quad (1a)$$

$$\text{subject to} \quad p_{\text{head}} \cdot z < p_{\text{obstacle}} \cdot z, \quad (1b)$$

$$\text{PoS} \in p_{\text{CoP}}, \quad (1c)$$

$$p_{\text{leftFoot}} \in \text{GND}, \quad (1d)$$

$$p_{\text{rightFoot}} \in \text{GND}, \quad (1e)$$

$$q_i^{\min} \leq q_i \leq q_i^{\max} \quad (1f)$$

The core of every optimization is the objective function. For dodging, stability is the metric we would like to optimize. Equation 1a describes the distance from the PoS centroid ( $C_{\text{PoS}}$ ) towards the CoP. If necessary, this objective function could be extended with other popular cost functions from humanoid optimal control. Examples would be minimization of effort (joint torque squared), maximization of manipulability, or minimization of angular momentum around the CoM. Because we are only concerned about finding the optimal dodge pose and not walking the robot towards it, static stability in combination with a simple calculation for the CoP covers our requirements.

While the centroid distance brings some advantages like ease of calculation and derivation, it also bears some problems. The PoS, however, is not approximately equally stretched into every dimension. Because we take the distance to the centroid as a metric, the form of the PoS only indirectly influences the metric. It does not consider that we have a lot more allowed body movements towards the feet than towards the front or back. The figure 1a displays this phenomenon.

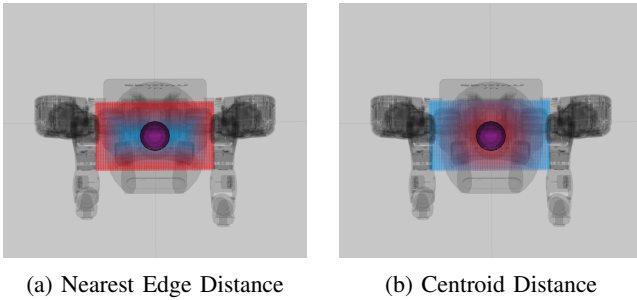


Fig. 1: Both images display the stability criteria inside the PoS. Red represents a low value in the respective criteria, while blue is a higher value.

The CoM is visualized as a purple sphere. The stability criteria are always calculated from the CoM and the PoS. For better comparability, a fixed PoS and the CoM in default pose are used.

An alternative would be the in figure 1b shown nearest-edge-distance. It is described by the formula

$$\tilde{s}(q) = \min_j \|e_j, p_{\text{CoP}}\|, \quad e_j : \text{edge of PoS}$$

and by its nature respects the extension of the polygon. The nearest-edge-distance in its core is not derivable, as it is using a minimum operator. There is extensive research on this topic. An example would be the smooth functions in R. S. Womersley et al. [22].

However, as the goal of this project is to present a simple explanatory example, the centroid distance is used as a stability metric.

In addition to the stability, another core constraint of the problem is to avoid the incoming obstacle. Equation 1b handles this and is therefore referred to as the dodge constraint. It ensures that the found pose is collision-free. The implementation in 1b is an example of such a simple constraint. Here the assumption is made that if the robot moves his head underneath the approaching height of the obstacle, the obstacle can be fully dodged. This assumes that either the obstacle flies in parallel to the ground or the height at which the obstacle would hit the robot is known. It is a crude simplification of the problem but well-suited towards a solution with an optimization algorithm and therefore represents a good starting point for this example.

Next up, the two equations 1d and 1e are set to keep the robot's feet on the ground. Otherwise the robot would try to lift its feet up and change its PoS. As it is our goal to keep the PoS fixed, foot motions must be restricted.

The last equation (eq. 1f) includes the limits for each joint. To ensure that the humanoid only finds poses that are reachable by the real robot, this constraint needs to be put into place.

In sum, the following generalized optimization problem can be formulated:

- 1) *Objective Function*: Stability Criteria (1a)
- 2) *Dodge Constraint* (1b)
- 3) *Stability Constraint* (1c)
- 4) *Foot-Ground Constraint* (1d & 1e)
- 5) *Joint-Limit Constraints* (1f)

The formalization is not specific to finding a pose but also can be generalized towards full optimal control. However constraints 3 to 5 need to be ensured during motion. Constraint 5 should also be extended for optimal control. Additionally torque and acceleration limitations would need to be put in place.

The *Dodge Constraint* (2) only has to be satisfied at a specific point in time. This exact moment, as well as the location, can be computed from any perception component the robot might include.

### III. DESIGN AND IMPLEMENTATION

This section focuses on displaying the whole system and framework in its current implementation. It tries to explain design, selected libraries, and the resulting architecture in a way that enables the reader to utilize the presented components or add new ones if necessary. The framework does not aim to provide an already perfect toolbox, but one that can be easily modified and tuned towards the reader's liking. It starts off with describing the general tools and technologies included. (III-A) Naturally this leads towards the interconnection between the different libraries and their roles inside

the entire dodging framework. (III-B) A brief explanation of how each component is used and could be employed in future works is also added. The section finishes off with the core of the framework. (III-C) To employ the strength of RBDL in combination with a simulation like Gazebo or RViz, a wrapper is needed.

### A. Tools

These tools offer a starting point for optimization-based solutions to the bespoke dodge problem. Machine learning-based solutions are therefore only sparsely supported but can be added by the user if such technology presents itself as necessary.

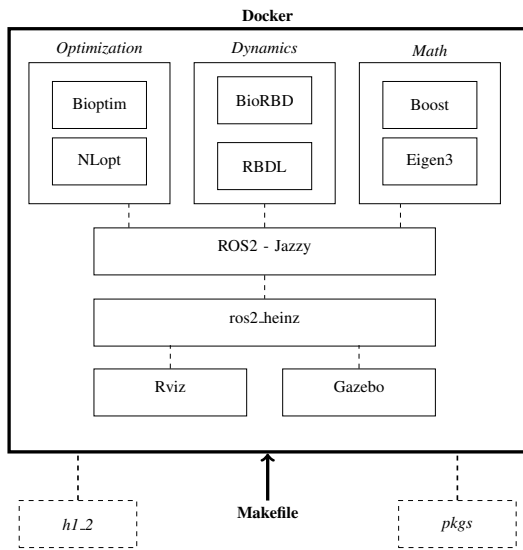


Fig. 2: The docker encapsulates the total ROS2 environment from the user’s operating system. Only the workspace *h1\_2* with all nodes and an additional packages folder *pkgs* is mounted to the system. Inside is simulation software installed like RViz and Gazebo. Those are connected using the *ros2.heinz* package to ROS2. Additionally, the Docker provides tools for optimization, robot dynamics, and connected math libraries.

Figure 2 displays all used technologies and how they interact with each other.

A main requirement for this system was portability and ease of setup. However, ROS2 is the de facto standard when it comes to robotics prototyping and interaction but presents numerous difficulties when setting up. The first hurdle is most of the time the operating system itself. Installing ROS2 on an Ubuntu machine is flawless, while installation on any different Linux distribution comes with multiple additional steps to take. And even if an Ubuntu machine is in use, it still can be the wrong version. Older ROS2 versions demand old Ubuntu versions and vice versa.

Therefore the decision was made to use Docker [23] as a solution. Docker containers encapsulate a virtual operating system while providing zero to none overhead to the underlying system. The container allows the user to have a portable setup. Docker containers come by default without support

for GPUs. To tackle this issue, a detection for Nvidia GPUs has been added. It enables the use and pass-through for the Docker container if such a GPU is available. As a tool to conveniently manage the docker, a makefile is provided. The

TABLE I: Custom Makefile-defined Docker commands

| Command | Description                                                                                                                                                                                                              |
|---------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| run     | launches the docker with <code>docker run</code> and enables X11-forwarding on the host machine                                                                                                                          |
| build   | create <code>ws/src/</code> and adjust permissions, calls <code>docker build</code> with the correct container name                                                                                                      |
| clean   | removes colcon artifacts in <code>ws</code> and deletes Docker from the internal list (this needs sudo privileges, because Docker had sudo permissions when creating those files) $\Rightarrow$ full build is necessary! |
| rebuild | combination of <code>clean</code> and <code>build</code> without the use of cache                                                                                                                                        |
| stop    | used to stop the docker when the process where <code>run</code> was called is not accessible (calls <code>docker stop</code> )                                                                                           |

full list of available commands can be seen in table I.

In most cases only the commands `run` and `build` are regularly used. The `run` command starts the docker with appropriate parameters and mounts two directories.

The folder *h1\_2* is the main ROS2 workspace. Here are all packages and build directories located. Mounting it to disc creates a persistent space that is not reset after each Docker shutdown. Another equivalent space is *pkgs*. It is meant to hold external packages. Because it is outside the ROS2 workspace and only linked inside it, external packages are easy to keep track of. An example, therefore, is *ros2.heinz*, which helps with controlling Unitree robots and especially the H1. To run the Docker, it has to build first. This is the function of `build`.

The already mentioned *ros2.heinz* package was built by Karsten Appenzeller. This bridge connects the simulation tool Gazebo with ROS2. It also contains the *urdf* file, which describes the robot’s body. A property that is necessary for using dynamics libraries such as RBDL and BioRBD[24].

The user is granted access to the three primary pillars of *Optimization*, *Dynamics*, and *Math* to establish a consistent foundation for optimization and optimal control-based research. They also build on top of each other.

ROS2 enables the user to switch between incorporating Python and C++ nodes side-by-side. With the goal of real-time calculations and object dodging in mind, C++ was chosen as a main programming language of this project. This also influenced the selection of used libraries. An overview of the compatibility of each chosen library is shown in table II.

The bases are the *Math* libraries Boost [25] and Eigen3 [26]. Normally it is recommended to keep a homogenous math foundation in the form of only one library. However, because ROS2 uses mainly Boost, while, for example, RBDL uses Eigen3, it is not possible to lock in only one single library. Both have strength in linear algebra and sometimes cover the same topics. However, due to its widespread use and modern C++ interface, Eigen3 is used for most packages in this project. Boost moves into more of a support role.

TABLE II: Interface Languages of the Libraries

| Library  | Languages     | Reference                     |
|----------|---------------|-------------------------------|
| Boost    | C++ / Python  | Gurtovoy, Aleksey et al. [25] |
| Eigen3   | C++           | Guennebaud, Gaël et al. [26]  |
| BioRBD   | C++* / Python | Michaud, Benjamin et al. [24] |
| RBDL     | C++           | Martin L. Felis [27]          |
| Bioptrim | C++* / Python | Benjamin, Michaud et al. [28] |
| NLopt    | C++ / Python  | Steven G. Johnson [29]        |

\*Library provides C++ code generation, but no native or partial support.

Providing functions like point-in-polygon checks that Eigen3 does not include.

Both *Optimization* and *Dynamics* build heavily on top of these two libraries. Next up is the pillar *Dynamics*, which includes RBDL and BioRBD. BioRBD is an extension of the popular rigid body dynamics library RBDL. While RBDL provides the general computations and capabilities for `urdf` file imports, BioRBD is more catered towards analyzing biomechanical data. The presented project only requires simple forward kinematics with a rigid body system, so RBDL alone is enough to calculate these.

Moving on to the last pillar, *Optimization*. It contains the libraries Bioptrim [28] and NLopt [29]. Bioptrim is built on top of NLopt and provides full optimal control functionalities. Stock NLopt, however, solves optimization problems itself and therefore requires the additional layer to provide an easy-to-use optimal control experience. Because this project just calculates a stable dodge pose and not the motion, NLopt already solves this problem.

### B. Architecture

While section III-A displayed the interactions and capabilities of included libraries, this section adds on top of these. Utilizing additional custom-made packages to enhance the research process. To put them to each other in perspective, figure 3 displays relations between the different packages. Inside the diagram, all base packages, which are concerned with ROS2, Gazebo, or the bridges in between, are encapsulated by *ros2\_heinz*.

Using this abstraction layer, the three packages *neo\_utils*, *point\_calculator*, and *dodge\_it* were created.

The utility package *neo\_utils* conceals common functionality away so that they can be reused consistently throughout different algorithms. *Stability*, for example, holds different stability criteria like minimal-edge-distance or centroid-distance. The *RBDLWrapper* improves the interaction with the RBDL library and keeps it in sync with the running simulation.

Next up is the *point\_calculator* package. It uses the given information from the simulation ROS2 topics to calculate important stability concepts. Two main ones are the CoM (*CenterOfMass*) and PoS (*PolygonOfSupport*). The package builds on top of the already existing *neo\_utils* package to enforce consistency across the project.

Of all packages, the *dodge\_it* package has the highest level of abstraction. It is powered by the previously described

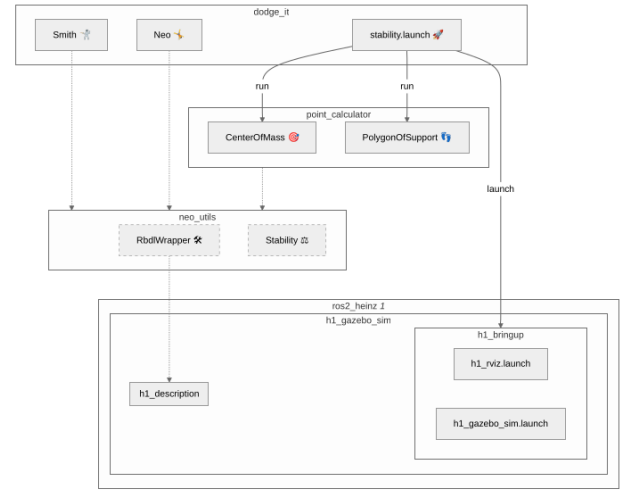


Fig. 3: The ROS2 structure of this project consists of four major packages. *ros2\_heinz* and *neo\_utils* are the backbone and provide functionality that most of the other packages use. Inside *point\_calculator* there are nodes to publish the PoS and CoM. The last package, which is called *dodge\_it*, holds implementations of the dodging algorithms. For convenience, the package is extended with a launch-file.

components and only holds dodge algorithms, as well as their launch files. In the case of this project, only *Smith* and *stability.launch* are implemented. The component *Neo* in figure 3 functions as a placeholder for further, more advanced implementations the reader might do. Launch files in general help the user to start multiple ROS2 nodes by executing a single file. The *stability.launch* file functions as an example for how to set up such a file. It implements a launch configuration for Gazebo and RViz, with and without simulation. The file launches necessary nodes such as *CenterOfMass* and *PolygonOfSupport*. For further information, take a look at the example file in the GitHub repository. (GitHub)

### C. RBDL Wrapper

To keep the dynamics model in sync with the robot's actual state (state represented in ROS2 topics), an additional helper is necessary. The following section is concerned with wrapping the RBDL library to accomplish this goal. Figure 4 displays the component of the spoken wrapper and how it can be accessed by the user. Moving along the figures structure, the *RBDLWrapper* encapsulates not only the dynamics library RBDL but also the ROS2 topic *joint\_states*. The topic is updated by a joint publisher, which is launched together with either Gazebo or RViz inside the provided launch file. It updates the selected visualization or simulation based on these joint states and publishes its state in this topic. Therefore it fully represents the robot state. Changes in the topic are synced towards the dynamics library. In the opposite direction, the changes are not instant. They need to be manually triggered by the user. This design decision was taken to enable the user to run multiple different computations before publishing the final

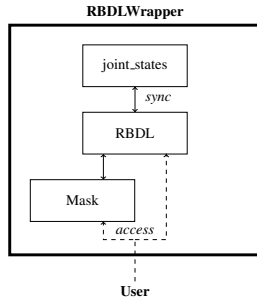


Fig. 4: The RBDLWrapper encapsulates the Rigid Body Dynamics Library (RBDL) library. It keeps RBDL updated with the current joint states of either the Gazebo simulation or RViz visualization. A mask adds a layer of abstraction to enable the user to ignore some joints without breaking RBDL.

For accessibility, the raw RBDL functions and objects can be accessed anytime.

joint states to the simulation. A process that is vital for optimization algorithms.

Another functionality specifically designed for optimization is the *Mask*. It is a bidirectional masking tool that lets the user fix certain joints in place. When doing optimal control or optimization-based problem solving, high DoF leads to difficult and redundant solution spaces. Reducing these variables helps to push the algorithms towards convergence. For the user, these joints are ignored when retrieving joint states. However there is still the option to unmask or mask an array of joint states and retrieve the full joint states.

A feature-rich framework to solve the dodge problem was presented. The following section shows how to use these components to solve a simple problem.

#### IV. EXAMPLE OPTIMIZATION

The following section describes how the dodge problem described in section II can be solved. Equation 1 was implemented during this project, utilizing the tools and components presented in section III.

First the example implementation of the *Smith* algorithm is described. (IV-A) Section IV-B discusses some challenges. Through the example, common pitfalls are shown. Some of the not only specific to solving a dodge problem, but also general optimization-based algorithms. The section also presents, how the provided tools can be used to avoid these problems.

##### A. Specification and Implementation

To display the capabilities of the presented toolbox, an example was implemented. Figure 3 marks it as *Smith*. The algorithm was fully implemented in C++ and can be found in the github repository.

The goal was to write a node that finds the dodge pose that is most stable. One of the key parts here is that only the pose should be calculated and not the motion towards it. Therefore

some simplifications can be made. First we have no need for a full simulation like Gazebo and can use the visualization tool RViz instead. Because we are not calculating the motion itself, Gazebo would have difficulties spawning the robot into the world with the correct state. This is just something the simulation was not designed to do. RViz instead lets the user spawn in the robot in any pose. However, because we stripped out the physics engine, we rely on the static stability criteria to signal if a pose is feasible.

The optimization tool NLOpt, with its implementation of the Bound Optimization BY Quadratic Approximation (BOBYQA) [30][31] algorithm, was chosen to solve the problem. BOBYQA has the advantage, that it is derivative free.

Using the in package *point\_calculator* bundled calculations and the centroid-distance from *neo\_utils* the objective function and constraints can be solved. In order to limit the DoF the left foot was completely masked out and the torso fixed to remove rotational difficulties. The arms are also fixed

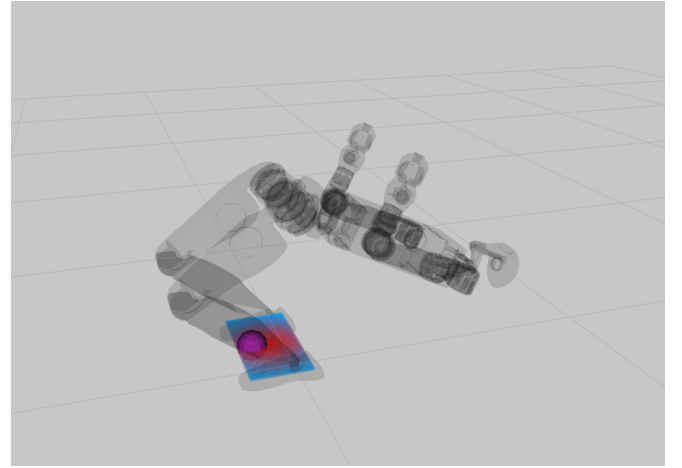


Fig. 5: After adding all constraints and executing the optimization, the above pose is found by the algorithm.

in place, so that effectively only the right leg and hip joint are flexible.

Together with a constant PoS that was predetermined the algorithm found the in figure 5 displayed pose.

##### B. Challenges and Discussion

In following some common challenges, when solving optimization problems with humanoids are showcased. These are problems that occurred while using the in section IV-A described implementation. To ease the reader into the topic this section describes the challenges and how they were overcome in the example implementation.

a) *Global Reference Frame*: Figure 6 displays a pose that was found early in development. The robot only tilts slightly forward. At this point the global reference frame was the root of the urdf file, which is the root of the robot. Because this coordinate frame is located in the pelvis area of the robot it is rigidly connected to the head, effectively breaking the constraint, which should pull the robots head

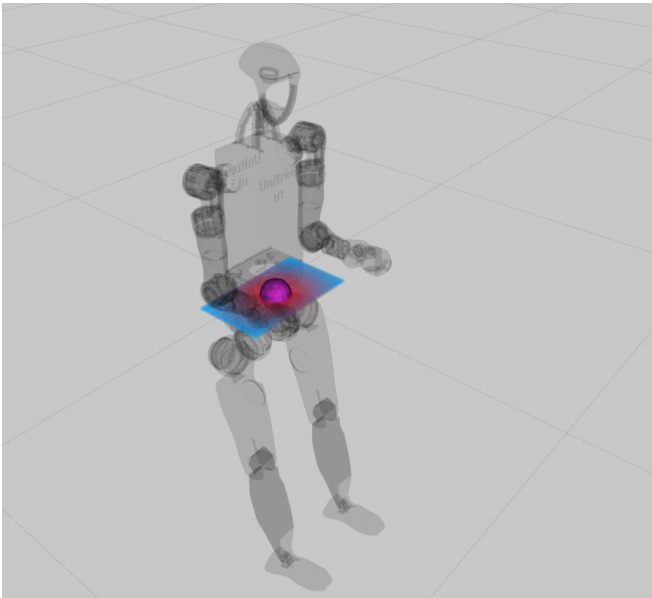


Fig. 6: The optimal control problem in II uses a world coordinate system as reference for all calculations. If selecting the root of the RBDL system as such, the algorithm won't converge and result in the shown pose. This is because there is no change in head-to-reference-frame distance. The root of the RBDL system is fixed together with the torso and therefore never changes its location relative to the head.

underneath a specific height.

When using Gazebo the option was there to use a global reference frame of the ground. But now that only RViz and no ground exists a different "world" coordinate system is needed. As an approximation an ankle link is chosen as new reference frame. This is possible due to the constraint, that the feet should all the time stay static on the ground. Otherwise the PoS would be changed.

*b) High DoF:* In section III-C the functionality was mentioned to mask out joints. This is an essential part of solving optimization problems, as high DoF result in divergence. During the implementation of the example such

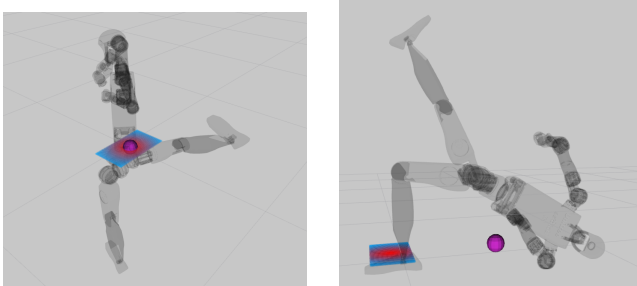


Fig. 7: These are two resulting poses, which showed up when using all DoF of the humanoid.

a problem showed up. Figure 7 displays the robot in different poses, which are either not stable or insufficient to the constraints. The most notable ignored constraint is that the robot has one feet in the air and therefore would change the PoS.

The *RBDLWrapper* is able to fix masked out joints in place and return them as constant values to RBDL. This results in NLopt only seeing and manipulating the unmasked joints. In the result image of figure 5 all but the leg and hip joint were fixed, so that the robot could not perform rotational motions. Additionally just the right leg was supplied to NLopt while the right one was mirrored before feeding the joint states into the RBDL.

*c) Initial Pose:* An important challenge to keep in mind is the initial starting configuration, when using an optimization-based algorithm. Figure 8 shows, how opti-

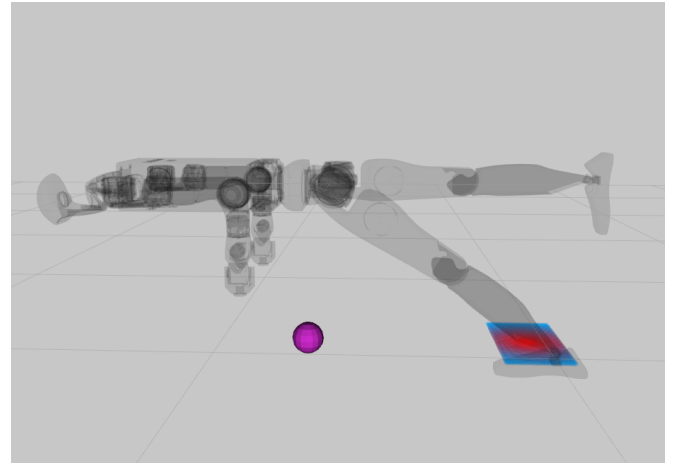


Fig. 8: This image shows what happens if the initial pose is not adjusted slightly towards a leaning position. The optimization algorithm cannot find a suitable minimum because the converged local minimum does not satisfy the stability constraint.

mization algorithms sometimes only find local optima, when given the wrong initial values.

In this case the starting position is the start joint configuration. Most of the time it is useful to put the robot as close to the goal position as possible. For this simple example the solution was to lean the robot some degree to the back, so that the optimization algorithm would push the robot backwards.

Solving optimization problems is not always straight forward. It is necessary to tweak initial conditions, limit variables or reformulate the problem to receive a converging solution.

## V. CONCLUSIONS AND OUTLOOK

The project showed that it is possible to build a simple algorithm for solving the discussed dodge problem. It provides a toolbox, which future research can build up on. Relying on a Docker container to enable fast and easy-to-use development, as well as including already well-established libraries for optimization and dynamics with additional custom packages. The example shows how a fully simulation- and optimization-focused approach can solve the difficult problem of humanoid dodging.

Through the already installed *Bioptim* library, full optimal control is the next logical step. A step that was not possible

to accomplish during the project because of time constraints. Another research option would be to move in the direction of dynamic PoS change. Something that Yoshida et al. [32] looked into and could be adjusted for a solution to the object dodge problem.

To further enhance the framework, more features would be interesting to look into. For example, incorporating simulations of camera images or LiDAR point clouds to fully simulate airborne objects. Another possibility would be to introduce abstracted obstacles that are simulated and provided to the algorithms through ROS2 topics. This would further encourage development towards real-time dodging and full optimal control.

As this section showed, the research field is still sparsely set but can be rapidly expanded with capable tools. Robots like the Unitree H1 enabled the research into dodge motions. Now it's time to move forward.

## VI. ACKNOWLEDGEMENT

Huge thanks to Karsten Appenzeller for providing the bridge and general abstraction layer to use the H1 inside Gazebo. His work can be found here.

*LanguageTool.org*<sup>1</sup> was used to improve the style of writing.

## REFERENCES

- [1] R. M. Schemschat, D. Clever, M. L. Felis, K. Mombaur, Optimal Push Recovery for Periodic Walking Motions, *IFAC-PapersOnLine* 49 (14) (2016) 93–98. doi:10.1016/j.ifacol.2016.07.991.  
URL <https://www.sciencedirect.com/science/article/pii/S2405896316312794>
- [2] J. Pratt, J. Carff, S. Drakunov, A. Goswami, Capture Point: A Step toward Humanoid Push Recovery, in: 2006 6th IEEE-RAS International Conference on Humanoid Robots, 2006, pp. 200–207, iSSN: 2164-0580. doi:10.1109/ICHR.2006.321385.  
URL <https://ieeexplore.ieee.org/document/4115602>
- [3] M. Cognetti, D. De Simone, L. Lanari, G. Oriolo, Real-time planning and execution of evasive motions for a humanoid robot, in: 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 4200–4206. doi:10.1109/ICRA.2016.7487614.  
URL <https://ieeexplore.ieee.org/document/7487614>
- [4] Z. Zou, K. Chen, Z. Shi, Y. Guo, J. Ye, Object Detection in 20 Years: A Survey, *Proceedings of the IEEE* 111 (3) (2023) 257–276. doi:10.1109/JPROC.2023.3238524.  
URL <https://ieeexplore.ieee.org/abstract/document/10028728>
- [5] A. Dastider, H. Fang, M. Lin, Unified Control Framework for Real-Time Interception and Obstacle Avoidance of Fast-Moving Objects with Diffusion Variational Autoencoder, in: 2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2024, pp. 13883–13890, iSSN: 2153-0866. doi:10.1109/IROS58592.2024.10802535.  
URL <https://ieeexplore.ieee.org/abstract/document/10802535>
- [6] S. Yu, S. Rammohan, K. Zheng, G. Konidaris, Hierarchical Reinforcement Learning of Locomotion Policies in Response to Approaching Objects: A Preliminary Study, *arXiv:2203.10616 [cs]* (Mar. 2022). doi:10.48550/arXiv.2203.10616.  
URL <http://arxiv.org/abs/2203.10616>
- [7] J. Koschorreck, K. Mombaur, Modeling and optimal control of human platform diving with somersaults and twists, *Optimization and Engineering* 13 (1) (2012) 29–56. doi:10.1007/s11081-011-9169-8.  
URL <https://doi.org/10.1007/s11081-011-9169-8>
- [8] N. Sanket, C. M. Parameshwara, C. Singh, A. V. Kuruttukulam, C. Fermüller, D. Scaramuzza, Y. Aloimonos, EVDodge: Embodied AI For High-Speed Dodging On A Quadrotor Using Event Cameras, 2019. doi:10.48550/arXiv.1906.02919.
- [9] N. Garg, N. Roy, Enabling Self-defense in Small Drones, in: *Proceedings of the 21st International Workshop on Mobile Computing Systems and Applications, HotMobile '20*, Association for Computing Machinery, New York, NY, USA, 2020, pp. 15–20. doi:10.1145/3376897.3377866.  
URL <https://dl.acm.org/doi/10.1145/3376897.3377866>
- [10] M. Lu, H. Chen, P. Lu, Perception and Avoidance of Multiple Small Fast Moving Objects for Quadrotors With Only Low-Cost RGBD Camera, *IEEE Robotics and Automation Letters* 7 (4) (2022) 11657–11664. doi:10.1109/LRA.2022.3205114.  
URL <https://ieeexplore.ieee.org/abstract/document/9881875>
- [11] O. Khatib, Real-time obstacle avoidance for manipulators and mobile robots, in: 1985 IEEE International Conference on Robotics and Automation Proceedings, Vol. 2, 1985, pp. 500–505. doi:10.1109/ROBOT.1985.1087247.  
URL <https://ieeexplore.ieee.org/document/1087247>
- [12] J. Siekmann, Y. Godse, A. Fern, J. Hurst, Sim-to-Real Learning of All Common Bipedal Gaits via Periodic Reward Composition, in: 2021 IEEE International Conference on Robotics and Automation (ICRA), 2021, pp. 7309–7315, iSSN: 2577-087X. doi:10.1109/ICRA48506.2021.9561814.  
URL <https://ieeexplore.ieee.org/document/9561814>
- [13] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, M. Hutter, Learning agile and dynamic motor skills for legged robots, *Science Robotics* 4 (26) (2019) eaau5872, publisher: American Association for the Advancement of Science. doi:10.1126/scirobotics.aau5872.  
URL <https://www.science.org/doi/10.1126/scirobotics.aau5872>
- [14] R. Deits, R. Tedrake, Footstep planning on uneven terrain with mixed-integer convex optimization, in: 2014 IEEE-RAS International Conference on Humanoid Robots, 2014, pp. 279–286, iSSN: 2164-0580. doi:10.1109/HUMANOIDS.2014.7041373.  
URL <https://ieeexplore.ieee.org/document/7041373>
- [15] M. Cognetti, D. De Simone, F. Patota, N. Scianca, L. Lanari, G. Oriolo, Real-time pursuit-evasion with humanoid robots, in: 2017 IEEE International Conference on Robotics and Automation (ICRA), 2017, pp. 4090–4095. doi:10.1109/ICRA.2017.7989470.  
URL <https://ieeexplore.ieee.org/document/7989470>
- [16] I. Kumagai, M. Morisawa, S. Nakaoka, F. Kanehiro, Efficient Locomotion Planning for a Humanoid Robot with Whole-Body Collision Avoidance Guided by Footsteps and Centroidal Sway Motion, in: 2018 IEEE-RAS 18th International Conference on Humanoid Robots (Humanoids), 2018, pp. 251–256, iSSN: 2164-0580. doi:10.1109/HUMANOIDS.2018.8624927.  
URL <https://ieeexplore.ieee.org/document/8624927>
- [17] H. Sugiura, M. Gienger, H. Janssen, C. Goerick, Real-time collision avoidance with whole body motion control for humanoid robots, in: 2007 IEEE/RSJ International Conference on Intelligent Robots and Systems, 2007, pp. 2053–2058, iSSN: 2153-0866. doi:10.1109/IROS.2007.4399062.  
URL <https://ieeexplore.ieee.org/document/4399062>
- [18] X. Cheng, Y. Ji, J. Chen, R. Yang, G. Yang, X. Wang, Expressive Whole-Body Control for Humanoid Robots, in: *Robotics: Science and Systems XX*, Robotics: Science and Systems Foundation, 2024. doi:10.15607/RSS.2024.XX.107.  
URL <http://www.roboticsproceedings.org/rss20/p107.pdf>
- [19] C. Sferrazza, D.-M. Huang, X. Lin, Y. Lee, P. Abbeel, Humanoid-Bench: Simulated Humanoid Benchmark for Whole-Body Locomotion and Manipulation, *arXiv:2403.10506 [cs]* (Jun. 2024). doi:10.48550/arXiv.2403.10506.  
URL <http://arxiv.org/abs/2403.10506>
- [20] E. Todorov, T. Erez, Y. Tassa, MuJoCo: A physics engine for model-based control, in: 2012 IEEE/RSJ International Conference

<sup>1</sup><https://languagetool.org>

- on Intelligent Robots and Systems, 2012, pp. 5026–5033, iSSN: 2153-0866. doi:10.1109/IROS.2012.6386109.  
URL <https://ieeexplore.ieee.org/document/6386109>
- [21] N. Koenig, A. Howard, Design and Use Paradigms for Gazebo, An Open-Source Multi-Robot Simulator, in: IEEE/RSJ International Conference on Intelligent Robots and Systems, Sendai, Japan, 2004, pp. 2149–2154.
- [22] R. S. Womersley, R. Fletcher, An algorithm for composite nonsmooth optimization problems, *Journal of Optimization Theory and Applications* 48 (3) (1986) 493–523. doi:10.1007/BF00940574.  
URL <https://doi.org/10.1007/BF00940574>
- [23] D. Merkel, Docker: Lightweight Linux Containers for Consistent Development and Deployment .
- [24] B. Michaud, M. Begon, biorbd: A C++, Python and MATLAB library to analyze and simulate the human body biomechanics, *Journal of Open Source Software* 6 (57) (2021) 2562. doi:10.21105/joss.02562.  
URL <https://joss.theoj.org/papers/10.21105/joss.02562>
- [25] A. Gurtovoy, D. Abrahams, THE BOOST C++ METAPROGRAMMING LIBRARY .
- [26] G. Guennebaud, B. Jacob, others, Eigen v3 (2010).  
URL <http://eigen.tuxfamily.org>
- [27] M. L. Felis, RBDL: an efficient rigid-body dynamics library using recursive algorithms, *Autonomous Robots* 41 (2) (2017) 495–511. doi:10.1007/s10514-016-9574-0.  
URL <https://doi.org/10.1007/s10514-016-9574-0>
- [28] B. Michaud, F. Bailly, E. Charbonneau, A. Ceglie, L. Sanchez, M. Begon, Bioptim, a Python Framework for Musculoskeletal Optimal Control in Biomechanics, *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 53 (1) (2023) 321–332. doi:10.1109/TSMC.2022.3183831.  
URL <https://ieeexplore.ieee.org/document/9808374>
- [29] S. G. Johnson, The NLOpt nonlinear-optimization package (2007).  
URL <https://github.com/stevengj/nlopt>
- [30] M. J. D. Powell, A direct search optimization method that models the objective and constraint functions by linear interpolation, in: S. Gomez, J.-P. Hennart (Eds.), *Advances in Optimization and Numerical Analysis*, Vol. 275 of *Mathematics and Its Applications*, Springer, 1994, pp. 51–67. doi:10.1007/978-94-015-8330-5\_4.
- [31] M. J. D. Powell, The BOBYQA algorithm for bound constrained optimization without derivatives, Tech. Rep. NA2009/06, Department of Applied Mathematics and Theoretical Physics, Cambridge University, Cambridge, UK (2009).
- [32] (PDF) Task-driven Support Polygon Reshaping for Humanoids, in: ResearchGate, 2025. doi:10.1109/ICHR.2006.321386.  
URL [https://www.researchgate.net/publication/224060462\\_Task-driven\\_Support\\_Polygon\\_Reshaping\\_for\\_Humanoids](https://www.researchgate.net/publication/224060462_Task-driven_Support_Polygon_Reshaping_for_Humanoids)

RViz ROS Visualization. 4–7

UAV Unmanned Aerial Vehicle. 2

## ACRONYMS

### BOBYQA

Bound Optimization BY Quadratic Approximation.  
6

CoM Center of Mass. 3, 5

CoP Center of Pressure. 3

DoF Degree Of Freedom. 2, 6, 7

LiDAR Light Detection and Ranging. 2, 8

### MuJoCo

Multi-Joint dynamics with Contact. 2

PoS Polygon of Support. 3, 5–8

RBDL Rigid Body Dynamics Library. 1, 2, 4–7

ROS2 Robot Operating System 2. 1, 2, 4, 5, 8